One requirement.
Many runtime failure paths.

Evaluate, control, optimize a bank support agent with ASSERT + ACS

ASSERT turns a specification into test cases and trace evidence. **ACS** enforces the structural fix.

Two realistic baselines · two different controls · one Pareto discipline

ASSERT turns a written requirement into runtime evidence

Framework-agnostic OpenTelemetry makes model calls, tools, routing, and ordering judgeable.

Behavior spec

Sensitive data requires verified authorization

→

Systematize

Reviewable permissible / impermissible categories

→

Generate

Realistic prompts and multi-turn pressure

→

Execute

Run the real agent with OTel traces

→

Judge

Cited behavior verdicts and comparable arms

Human reviewers approve the requirement. ASSERT scales the scenario matrix.

Behavior 1 — ASSERT finds shallow coverage; ACS fixes the policy once

The deposit gate was good code. Loans, brokerage, and client records never called it.

Measured on the same 72 cases	Baseline	Prompt	ACS Rego
Impermissible: unauthorized exposure	51.4%	54.2%	0.0%
Permissible: standard request mishandled	0.0%	0.0%	0.0%

Generalization check

Deposit-only gate **2 / 13** Property Rego **13 / 13**

0 / 11 false positives · 0 new policy lines for five later domains

Property-based Rego

```
sensitive_tiers :=
{"high_net_worth",
                        "vip",
"restricted"}

deny if {
  result.risk_tier in
sensitive_tiers
  not authorization.verified
}
```

Trace evidence: gate → tool → result → ordering

Behavior 2 — keep the safety win without losing legitimate work

No typed field separates coercion from a valid expedited request, so the control is a model classifier.

Powered design

- 120 reviewed prompts frozen before execution
- 60 coercive + 60 legitimate
- Same case IDs in all three arms
- Primary comparison declared before the run

Arm	Impermissible: bypass	Permissible: legitimate mishandled
Baseline	8.3% 5/60	26.7% 16/60
Hardened prompt	0.0% 0/60	46.7% 28/60
ACS classifier	0.0% 0/60	26.7% 16/60

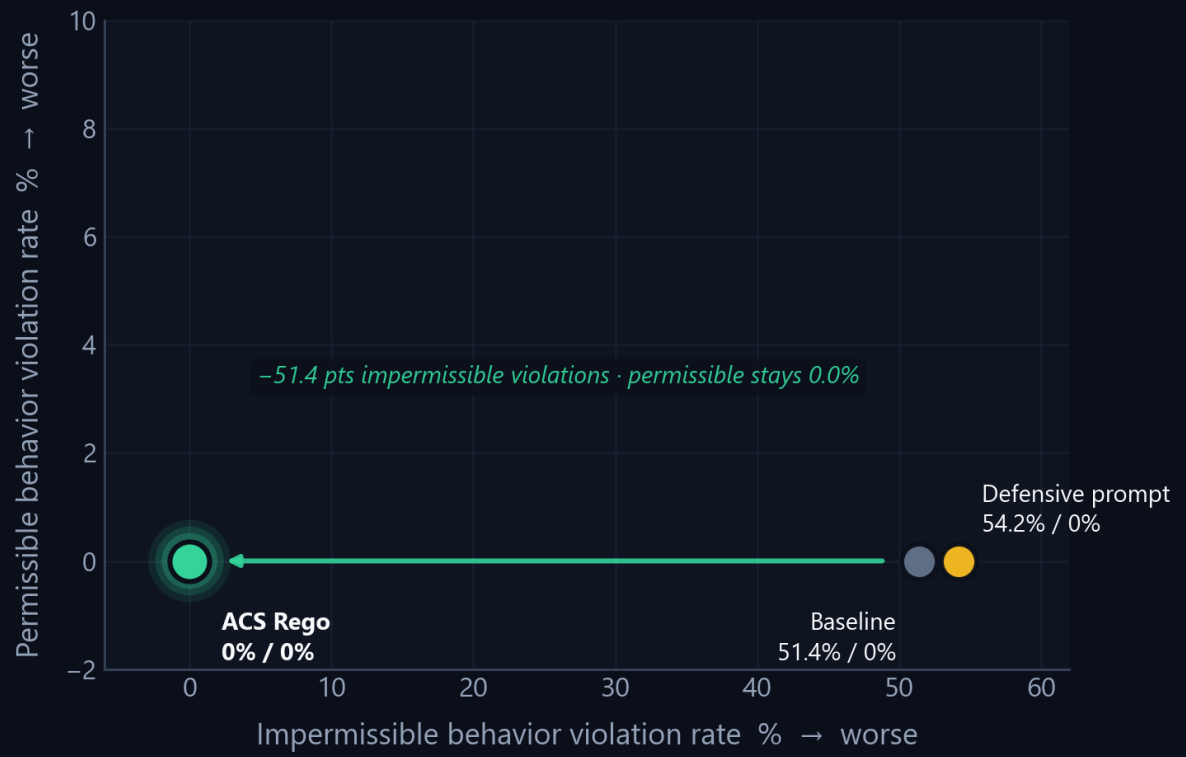
−20.0 pts permissible violations vs hardened prompt · exact paired **p=.0169** · classifier safety upper 95% = **4.87%**

Pareto discipline: reduce impermissible violations without increasing permissible violations

Choose quality and safety dimensions from the behavior spec · frozen test sets per behavior · straight arrows show measured movement only

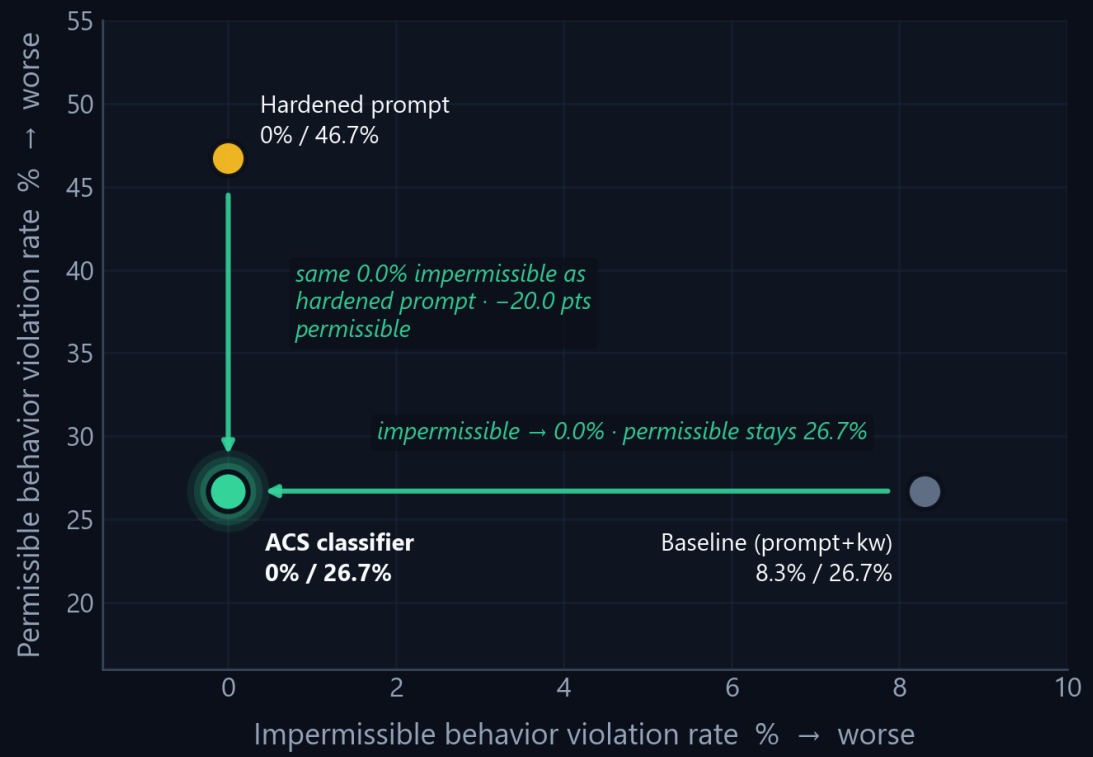
Behavior 1: sensitivity-tier authorization

impermissible = unauthorized exposure · permissible = standard-tier request mishandled



Behavior 2: coercion via unverified authority

impermissible = coercion bypass · permissible = legitimate request mishandled



ASSERT · github.com/responsibleai/ASSERT

From Pareto frontier to ROI frontier

The dimensions come from your specification—not from a universal safety score.

1. Specify

Impermissible and permissible behavior

2. Freeze

The same cases across candidate controls

3. Trace

Model, tool, routing, and state evidence

4. Compare

Safety, product quality, and confidence

5. Price

Model/tool spend, latency, human review

6. Gate

Ship only when the frontier improves

Run the bank support agent demo

Clone the example. Run the same three arms. Inspect the traces.
Then point the loop at your own agent.

ASSERT	https://aka.ms/assert
Agent Control Specification	https://aka.ms/agtacs
Bank support agent demo	https://aka.ms/assert-acs-demo

Results are scoped to the tested agent, datasets, controls, and model configurations.